Introducción Parte 3

Miembros estáticos y no estáticos

El miembro de una clase se divide en dos categorías.

Miembros estáticos

Los miembros (***variables, constructores y métodos***) que se crean utilizando el modificador estático se denominan miembros estáticos, no requieren una instancia

Miembros no estáticos el resto, requieren una instancia de una clase tanto para la inicialización como para la ejecución.

Variables estáticas y no estáticas

Siempre que declaramos una variable usando el modificador static o cuando declaramos una variable dentro de cualquier bloque estático, esas variables se consideran variables estáticas mientras que el resto de las demás se consideran variables no estáticas.

```
Public class Program
{
	int x = 10 ;// var no estática
	static int y  = 20 //var estática

	public static Main()// Bloque estático
	{
		int a = 10; // por defecto estática
	}

	void Metodocuaquiera()//Bloque no estatica
	{
		int b = 10; //por defecto no estática
	}

}
```

Variables estáticas y no estáticas

- La variable estática se inicializa inmediatamente una vez que comienza la ejecución de la clase,
- Las variables no estáticas se inicializan solo después de crear el objeto de la clase
- Una variable estática se inicializa solo una vez durante el ciclo de vida de una clase
- Una variable no estática se inicializa 0 o n veces, dependiendo de la cantidad de objetos creados para esa clase.
- Si desea acceder a los miembros estáticos de una clase, debe acceder a ellos directamente dentro de la misma clase y usar el nombre de la clase desde fuera de la clase,
- Necesita una instancia de una clase para acceder a los miembros no estáticos.

¿Cuál es el alcance de las variables estáticas y no estáticas en C#?

Las variables no estáticas se crean cuando se crea el objeto y se destruyen cuando se destruye el objeto.

El objeto se destruye cuando su variable de referencia se destruye o se inicializa con nulo.

Entonces, podemos decir que el alcance del objeto es el alcance de sus variables referenciadas.

Por otro lado, el alcance de la variable estática es el alcance de la aplicación cuando inicia la ejecución de la clase se crean y estarán allí mientras la aplicación se esté ejecutando.

Entonces, el alcance es el alcance de la aplicación.

Métodos estáticos y no estáticos en C#

No puede consumir miembros no estáticos directamente dentro de un método estático.

Si desea consumir miembros no estáticos con un método estático necesita crear un objeto

Puedes consumir directamente los miembros estáticos dentro de un método no estático sin ninguna restricción.

Reglas a seguir al trabajar con miembros estáticos y no estáticos en C#:

De no estático a estático: los miembros *no estáticos* sólo se pueden consumir utilizando el objeto de esa clase dentro de un bloque estático.

De estático a estático: los miembros *estáticos* se pueden consumir directamente (dentro de la misma clase) o utilizando el nombre de la clase (desde fuera de la clase) dentro de otro bloque estático.

De Estático a no estático: los miembros *estáticos* se pueden consumir directamente o utilizando el nombre de la clase dentro de un bloque no estático.

De no estático a no estático: los miembros *no estáticos* se pueden consumir directamente o utilizando la palabra clave **"this"** dentro de otro bloque no estático.

Ejemplo

```
using System;
namespace DemoEstaticoNoEstatico
  {
    class Example
    {
        int x = 100;
        static int y = 200;

//Este es un bloque estático  *1
        static void Suma()
            {
            Example obj =  new Example();
            Console.WriteLine("Suma de 100
  y  200 es :" + (obj.x + y));
            Console.WriteLine("Suma de 100
  y 200 es :" + (obj.x + Example.y));
        }
//Este es un método no estático *2
 void Multi()
        {
            Console.WriteLine("Multiplicar 100 y
200 da:" + (this.x * Example.y));
            Console.WriteLine("Multiplicar 100 y
200 da:" + (x * y));
        }
//El método principal es un método estático *3

        static void Main(string[] args)
        {
// Llamar al método Agregar usando el nombre de
la clase
            Example.Suma();
            Add();//Llamar al método Mul dir/

 // Estático a no estático *4
            Example obj =  new Example();
//llamar al método Mul usando una instancia
            obj.Multi();
Console.ReadLine();
        }
    }
}
```

Constructores estáticos y no estáticos en C#

Constructor estático : Si creamos el constructor explícitamente mediante el modificador static

Constructor no estáticos: los demás.

El punto más importante a recordar es que **el constructor estático es el primer bloque de código que se ejecuta** en una clase.

No importa cuántos objetos haya creado para la clase, el constructor estático se ejecuta **solo una vez**.

Un constructor *no estático* se ejecuta sólo cuando creamos el objeto de la clase

No es posible crear un constructor estático con parámetros.

Ejemplo

```
using System;
namespace ConstructorEstaticoNoEstatico
{
    class Ejemplo
    {
        //Constructor estático *1
        static Ejemplo()
        {
            Console.WriteLine("Se llama al constructor estático");
        }
        //Constructor NO estático *2
        public Ejemplo()
        {
            Console.WriteLine("Se llama al constructor NO estático");
        }

        static void Main(string[] args) *3
            Console.WriteLine("Inicio");
            Example obj1 = new Ejemplo();
            Example obj2 = new Ejemplo();
            Console.WriteLine("Fin");
            Console.ReadLine();
        }
    }
}
```

Clase Estática en c#

La clase que se crea utilizando el modificador static se denomina clase estática en C#.

Una clase estática sólo puede contener miembros estáticos.

No es posible crear una instancia de una clase estática. Dado que contiene miembros estáticos.

Podemos acceder a los miembros estáticos de una clase usando el nombre de la clase.

Ejemplo

```
using System;
namespace EstaticoNoEstatico
{
    public static class ConvertidorTemperatura
    {
        public static double CelsiusAFahrenheit(string tempCelsius)
        {
            // Convertir argumento a doble para cálculos.
            double celsius = Double.Parse(tempCelsius);
            // Convierte grados Celsius a Fahrenheit.
            double fahrenheit = (celsius * 9 / 5) + 32;
            return fahrenheit;
        }
        public static double FahrenheitACelsius(string tempFahrenheit)
        {
            // Convertir argumento a doble para cálculos.
            double fahrenheit = Double.Parse(tempFahrenheit);
            // Convierte grados Fahrenheit a Celsius.
            double celsius = (fahrenheit - 32) * 5 / 9;
            return celsius;
        }
    }
```

Ejemplo cont

```
class ConvertirTemperatura
    {
        static void Main()
        {
            Console.WriteLine("Seleccione la Conversión");
            Console.WriteLine("1. Para Celsius a Fahrenheit.");
            Console.WriteLine("2. Para Fahrenheit a Celsius.");
            Console.Write(":");
            string selection = Console.ReadLine();
            double F, C = 0;
            switch (selection)
            {
                case "1":
                    Console.Write("Ingrese la temperatura en Celsius: ");
                    F = ConvertidorTemperatura.CelsiusAFahrenheit(Console.ReadLine());
                    Console.WriteLine(" La Temperatura en Fahrenheit: {0:F2}", F);
                    break;
                case "2":
                    Console.Write("Ingrese la temperatura en Fahrenheit: ");
                    C = ConvertidorTemperatura.FahrenheitACelsius(Console.ReadLine());
                    Console.WriteLine(" La Temperatura en  Celsius: {0:F2}", C);
                    break;
```

Ejemplo cont.

```
                default:
                    Console.WriteLine("Seleccione la opción 1 o 2.");
                    break;
            }
            // Keep the console window open in debug mode.
            Console.WriteLine("Presione una tecla.");
            Console.ReadKey();
        }
    }
}
```

Constantes y Solo Lectura en C#

Las *variables constantes* son los valores inmutables que se conocen en el momento de la compilación del programa y no cambian sus valores durante la vida útil del programa.

Las *variables de sólo lectura* también son valores inmutables, pero estos valores se conocen en tiempo de ejecución y tampoco cambian sus valores durante la vida del programa.

Constantes en C#:

Declaramos una variable usando la palabra clave const. Si o si se debe inicializar.

```
const float PI = 3.14f;
```

sino

```
const float PI;
```



Las constantes son estáticas por defecto.

Constante = Variable Estática *1

La única diferencia, las variables estáticas se pueden modificar.

Variables de sólo lectura en C#

Declaramos una variable usando la palabra clave readonly

No es obligatorio inicializar una variable *readonly* en el momento de su declaración.

No se pueden modificar después de la inicialización.

Se pueden inicializar bajo el constructor, podemos modificar el valor de la variable *readonly* dentro de un constructor.

Para acceder a variables *readonly* necesitamos una instancia.

La diferencia entre una variable const y una readonly en C# es que una constante es un valor **fijo** para **toda la clase**, mientras que readonly es un valor **fijo** específico **de una instancia** de una clase y para cada instancia

Ejemplo

```
using System;
namespace ConstantesYsoloLecturaENE
{
    internal class Program
    {
        const float PI = 3.14f;
        static int x = 100;
        int y;
        readonly int z;

        //Constructor
        public Program(int a, int b)
        {
      //Iniciando variable no static
            y = a;
      //Inicializando variable readonly
            z = b;
        }
```

```
 static void Main(string[] args)
        {
//Accediendo a la variable estática sin
instancia
            Console.WriteLine($"x value: {x}");
//Accediendo a la variable const sin instancia
            Console.WriteLine($"PI value: {PI}");

//Creando 2 instancias
            Program obj1 = new Program(300, 45);
            Program obj2 = new Program(400, 55);

//Acceso a variables no static y de readonly
mediante instancia
            Console.WriteLine($"valor obj1 y:
{obj1.y} y valor readonly z: {obj1.z}" );
            Console.WriteLine($"valor obj2 y:
{obj2.y} y  valor readonly z: {obj2.z}" );
            Console.Read();
        }
    }
}
```

Realice los ejercicios 1 al 4 de la guia de trabajo

Propiedades

Son un tipo especial de miembros de clase que proporcionan un mecanismo flexible para leer, escribir o calcular el valor de un campo privado.

Se pueden utilizar como si fueran miembros de datos públicos, pero en realidad son métodos especiales llamados **“accesores”** .

Accesores: El bloque de “set” y “get” se conoce como “Accessors”.

Es muy importante restringir la accesibilidad a la propiedad.

Hay dos tipos de descriptores de acceso:

- obtener descriptores de acceso
- establecer descriptores de acceso .

Tipos de propiedades basadas en los descriptores de acceso "get" y "set":

- **Propiedades de lectura y escritura:** cuando la propiedad contiene métodos get y set.
- **Propiedades de solo lectura:** cuando la propiedad contiene solo el método get.
- **Propiedades de solo escritura:** cuando la propiedad contiene solo el método establecido.
- **Propiedades implementadas automáticamente:** cuando no hay lógica adicional en los descriptores de acceso a la propiedad y se introduce en C# 3.0.

Ventajas de utilizar Propiedades en C#

1. Las propiedades proporcionarán la abstracción de los campos de datos

2. También brindan seguridad a los campos de datos

3. Las propiedades también pueden validar los datos antes de almacenarlos en los campos de datos.

Accesores simétricos y asimétricos en C#

Simétrico Tanto get como set tiene la misma accesibilidad

```
public int EmpId
{
    set { _EmpId = value; }
    get {  return _EmpId; }
}
```

Asimétrico si no son los mismos

```
public int EmpId
{
    protected set { _EmpId = value; }
    get {  return _EmpId; }
}

public int EmpId
{
    private set { _EmpId = value; }
    get {  return _EmpId; }
}

public int EmpId
{
    set { _EmpId = value; }
    private get {  return _EmpId; }

}
```

Propiedades vs Getter y Setter

Existen lenguajes de programación que no tienen propiedades conceptuales y dichos lenguajes de programación utilizan métodos getter y setter para encapsular y proteger campos

La ventaja de las propiedades sobre los métodos tradicionales getter() y setter() es que podemos acceder a ellas ya que son campos públicos, no métodos.